



योग: कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

VIVEKANANDA INSTITUTE OF PROFESSIONAL STUDIES - TECHNICAL CAMPUS

Grade A++ Accredited Institution by NAAC

NBA Accredited for MCA Programme; Recognized under Section 2(f) by UGC;
Affiliated to GGSIP University, Delhi; Recognized by Bar Council of India and AICTE
An ISO 9001:2015 Certified Institution

SCHOOL OF ENGINEERING & TECHNOLOGY

**BTECH Programme: Artificial Intelligence and
Machine Learning – (A) (6th Semester)**

Course Title: Digital Image Processing Lab

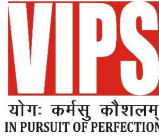
Course Code: AIML302P

Submitted To:

Ms. Princy Aggarwal

Submitted By:

**Name: Kunsh Sabharwal
Enrolment No: 01117711623**



VIVEKANANDA INSTITUTE OF PROFESSIONAL STUDIES - TECHNICAL CAMPUS

Grade A++ Accredited Institution by NAAC

NBA Accredited for MCA Programme; Recognized under Section 2(f) by UGC;
Affiliated to GGSIP University, Delhi; Recognized by Bar Council of India and AICTE
An ISO 9001:2015 Certified Institution

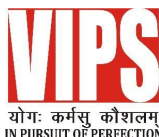
SCHOOL OF ENGINEERING & TECHNOLOGY

VISION OF INSTITUTE

To be an educational institute that empowers the field of engineering to build a sustainable future by providing quality education with innovative practices that supports people, planet and profit.

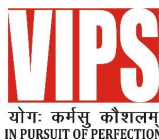
MISSION OF INSTITUTE

To groom the future engineers by providing value-based education and awakening students' curiosity, nurturing creativity and building capabilities to enable them to make significant contributions to the world.



VIVEKANANDA INSTITUTE OF PROFESSIONAL STUDIES - TECHNICAL CAMPUS
Grade A++ Accredited Institution by NAAC
 NBA Accredited for MCA Programme; Recognized under Section 2(f) by UGC;
 Affiliated to GGSIP University, Delhi; Recognized by Bar Council of India and AICTE
 An ISO 9001:2015 Certified Institution
SCHOOL OF ENGINEERING & TECHNOLOGY

S. No.	Experiment	Date	Marks (Total Five) which includes (Laboratory Assessment, Class participation and Viva)	Remarks	Updated Marks	Faculty Signature
1.	Create a program to demonstrate Geometric transformations – Image rotation, scaling, and translation.					
2.	Display of FFT (1-D & 2-D) of an image and apply Two-dimensional Fourier transform using DFT and masking with DFT.					
3.	Write a program for Contrast stretching of a low contrast image, Histogram, Histogram Equalization, and display of bit planes of an image.					
4.	Computation of Mean, Standard Deviation, Correlation coefficient of the given image.					



VIVEKANANDA INSTITUTE OF PROFESSIONAL STUDIES - TECHNICAL CAMPUS
Grade A++ Accredited Institution by NAAC
 NBA Accredited for MCA Programme; Recognized under Section 2(f) by UGC;
 Affiliated to GGSIP University, Delhi; Recognized by Bar Council of India and AICTE
 An ISO 9001:2015 Certified Institution
SCHOOL OF ENGINEERING & TECHNOLOGY

5.	Implementation of Image Smoothing Filters (Mean and Median filtering of an image).					
6.	Implementation of Image sharpening filters and Edge Detection using Gradient Filters.					
7.	Implementation of Image Compression by DCT, DPCM, HUFFMAN coding.					
8.	Implementation of image restoring techniques.					
9.	Implementation of Image Intensity slicing technique for image enhancement.					
10.	Study and implement Canny edge detection algorithms on images and compare it with existing edge detection algorithms.					

EXPERIMENT 1

PROBLEM STATEMENT: Create a program to demonstrate geometric transformations – Image rotation, scaling, and translation.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Geometric transformations are fundamental operations in digital image processing that modify the spatial arrangement of pixels within an image without altering their intensity values. These transformations are widely used in applications such as computer vision, medical imaging, remote sensing, and computer graphics. The most used geometric transformations include translation, scaling, and rotation, which allow images to be repositioned, resized, and reoriented in a controlled manner.

A geometric transformation maps the coordinates of an input image to new coordinates in an output image using mathematical models, usually in the form of transformation matrices. These operations help in image alignment, normalization, and preprocessing for higher-level image analysis tasks.

Image Translation: - Image translation is the process of shifting an image from one position to another along the horizontal (x-axis) and vertical (y-axis) directions. In this transformation, every pixel in the image is moved by a fixed distance without changing its orientation or size. Translation is represented using a translation matrix that adds displacement values to the original pixel coordinates. This operation is commonly used to adjust image positioning or to correct misalignment between images.

Image Scaling: - Image scaling is a geometric transformation that changes the size of an image by enlarging or reducing it. Scaling can be performed either uniformly or non-uniformly along different axes. Image reduction, also known as subsampling, decreases the image size by selecting or averaging pixel values, while image enlargement (zooming) increases the image size using pixel replication or interpolation techniques. Scaling is frequently used to modify the resolution of an image, reduce storage requirements, or prepare images for multi-scale processing. It is considered a special case of affine transformation.

Image Rotation: - Image rotation involves rotating an image by a specified angle around a fixed point, typically the centre of the image. This transformation is achieved by multiplying the pixel coordinates with a rotation matrix that depends on the angle of rotation. Rotation is useful in correcting image orientation, pattern recognition, and object alignment tasks. The shape and structure of the image are preserved, while its orientation changes.

In conclusion, geometric transformations play a crucial role in image processing by enabling flexible manipulation of image geometry. Translation, scaling, and rotation serve as basic building blocks for more advanced image processing and computer vision algorithms.

CODE SCREENSHOTS:

```
[3]
✓ 14s ▶ import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()
image_path = list(uploaded.keys())[0]

img = cv2.imread(image_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

rows, cols, _ = img.shape

tx, ty = 50, 30
Translation_matrix = np.float32([[1, 0, tx],
                                  [0, 1, ty]])
translated_img = cv2.warpAffine(img, Translation_matrix, (cols, rows))

scaled_img = cv2.resize(img, None, fx=0.5, fy=0.5)

angle = 45
center = (cols // 2, rows // 2)
R = cv2.getRotationMatrix2D(center, angle, 1)
rotated_img = cv2.warpAffine(img, R, (cols, rows))

plt.figure(figsize=(12, 8))
```

```
[3] ✓ 14s ▶ plt.subplot(2, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

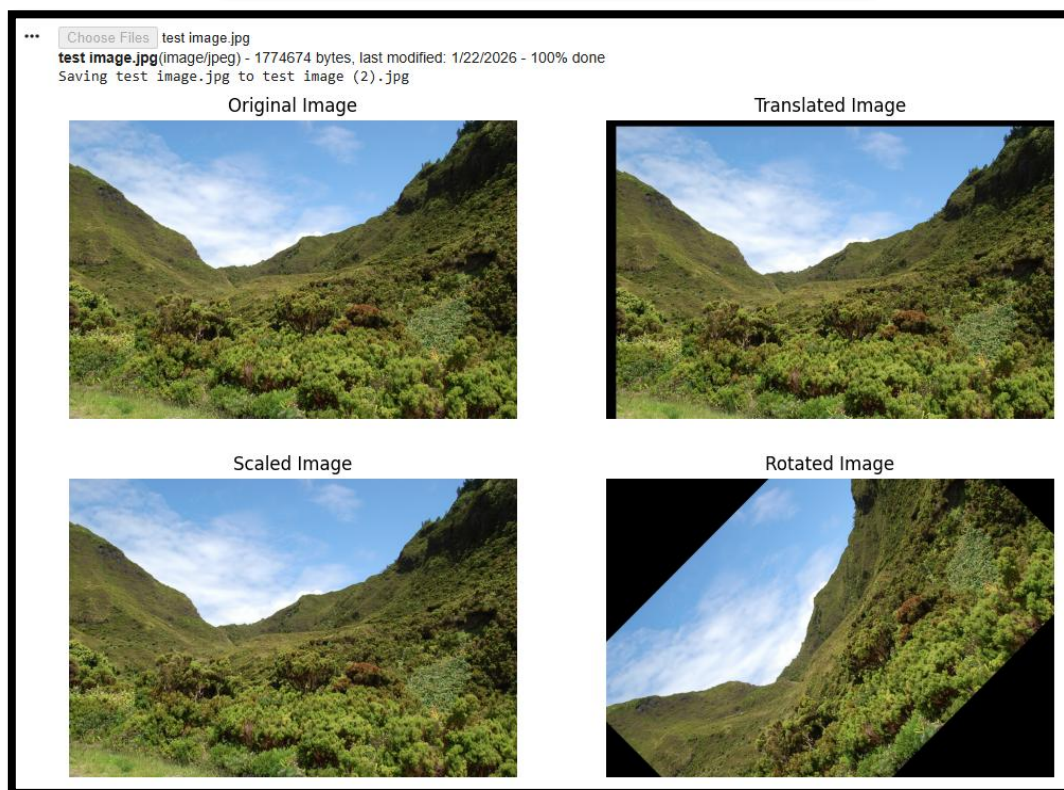
plt.subplot(2, 2, 2)
plt.imshow(translated_img)
plt.title("Translated Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(scaled_img)
plt.title("Scaled Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(rotated_img)
plt.title("Rotated Image")
plt.axis("off")

plt.show()
```

OUTPUT:



LEARNING OUTCOME

EXPERIMENT 2

PROBLEM STATEMENT: Display of FFT (1-D & 2-D) of an image and apply Two-dimensional Fourier transform using DFT and masking with DFT.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

In digital image processing, the Fourier Transform serves as an essential mathematical technique for analyzing the frequency characteristics of an image. Normally, images are interpreted in the spatial domain, where pixel values represent intensity variations across rows and columns. However, the Fourier Transform converts this spatial representation into the frequency domain, revealing how different frequency components contribute to the overall image.

Within the frequency domain, an image can be expressed as a combination of sinusoidal functions with varying frequencies and amplitudes. This representation is particularly useful for applications such as image enhancement, noise reduction, compression, and edge analysis, as it enables operations based on patterns and structures rather than individual pixels.

1. One-Dimensional and Two-Dimensional Fast Fourier Transform (FFT)

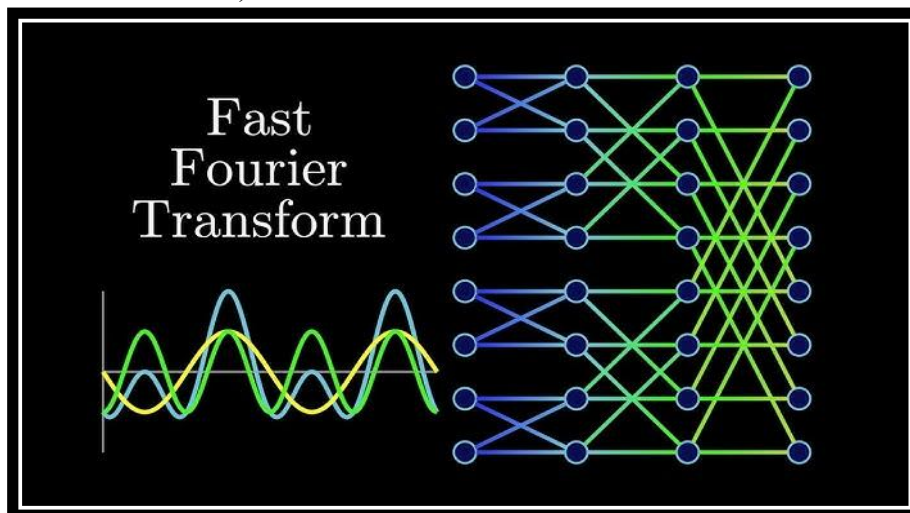
The Fast Fourier Transform (FFT) is an optimized computational method used to efficiently evaluate the Discrete Fourier Transform (DFT).

- 1-D FFT:

A one-dimensional FFT is commonly applied to signals that vary along a single axis, such as time-based audio signals. In image processing, a 1-D FFT can be applied to a single row or column of an image to study the variation of pixel intensity along that direction.

- 2-D FFT:

Since images consist of two-dimensional data, a two-dimensional FFT is more appropriate. The 2-D FFT analyzes frequency variations along both horizontal and vertical directions simultaneously. The output is known as the frequency spectrum, where low-frequency components (representing smooth and slowly varying regions) appear near the centre, while high-frequency components (corresponding to edges, fine details, and noise) are located farther from the centre.



2. Discrete Fourier Transform (DFT) Representation of an Image

The Discrete Fourier Transform represents an image as a sum of complex exponential functions. Applying the DFT to an image results in a complex-valued frequency representation, which is generally analyzed through two key components:

- **Magnitude Spectrum:**

The magnitude spectrum indicates the strength or contribution of each frequency component. To enhance visibility of higher frequencies, a logarithmic transformation is typically applied, making subtle frequency details easier to observe.

- **Phase Spectrum:**

The phase spectrum encodes spatial information about the image. It plays a critical role in preserving the structural layout and shapes within the image, often having a greater impact on image reconstruction than magnitude alone.

By performing a frequency shift operation (commonly referred to as FFT shift), the zero-frequency component is relocated to the centre of the spectrum, resulting in a more interpretable and symmetric frequency representation.

3. Frequency Domain Masking Using DFT

Masking in the frequency domain involves selectively modifying specific frequency components of an image. This process is equivalent to filtering in the spatial domain and is achieved by multiplying the Fourier-transformed image with a predefined mask. Common types of frequency-domain filters include:

- **Low-Pass Filter (LPF):**

This filter allows low-frequency components to pass while suppressing high-frequency components. The resulting image appears smoother and blurred, effectively reducing noise and fine details.

- **High-Pass Filter (HPF):**

A high-pass filter blocks low frequencies and preserves high frequencies. This enhances edges and sharp transitions, making it useful for feature detection and edge enhancement.

- **Band-Pass Filter:**

This filter permits only a specific range of frequencies to pass, while attenuating both low and high frequencies outside the selected band.

After applying the desired mask in the frequency domain, the Inverse FFT is performed to convert the filtered frequency data back into the spatial domain, allowing the processed image to be visually analyzed.

CODE SCREENSHOTS:

[8]

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import cv2

def load_and_preprocess_image(path, size=(256, 256)):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        raise FileNotFoundError("Image not found. Check file path.")

    img = cv2.resize(img, size)
    img = img.astype(np.float32) / 255.0
    return img

def create_circular_mask(h, w, radius):
    center = (w // 2, h // 2)
    Y, X = np.ogrid[:h, :w]
    distance = np.sqrt((X - center[0])**2 + (Y - center[1])**2)
    mask = distance <= radius
    return tf.cast(mask, tf.complex64)

image_path = '/content/test_image.jpg'
image = load_and_preprocess_image(image_path)

image_tensor = tf.convert_to_tensor(image, dtype=tf.complex64)

fft_image = tf.signal.fft2d(image_tensor)
fft_shifted = tf.signal.fftshift(fft_image)

magnitude_spectrum = tf.math.log(1 + tf.abs(fft_shifted))
```

```

rows, cols = image.shape
lp_mask = create_circular_mask(rows, cols, radius=30)

filtered_fft = fft_shifted * lp_mask
masked_spectrum = tf.math.log(1 + tf.abs(filtered_fft))

ifft_shifted = tf.signal.ifftshift(filtered_fft)
reconstructed = tf.signal.ifft2d(ifft_shifted)
reconstructed = tf.abs(reconstructed)

plt.figure(figsize=(12, 10))

plt.subplot(2, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image (Spatial Domain)')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('Magnitude Spectrum (2-D FFT)')
plt.axis('off')

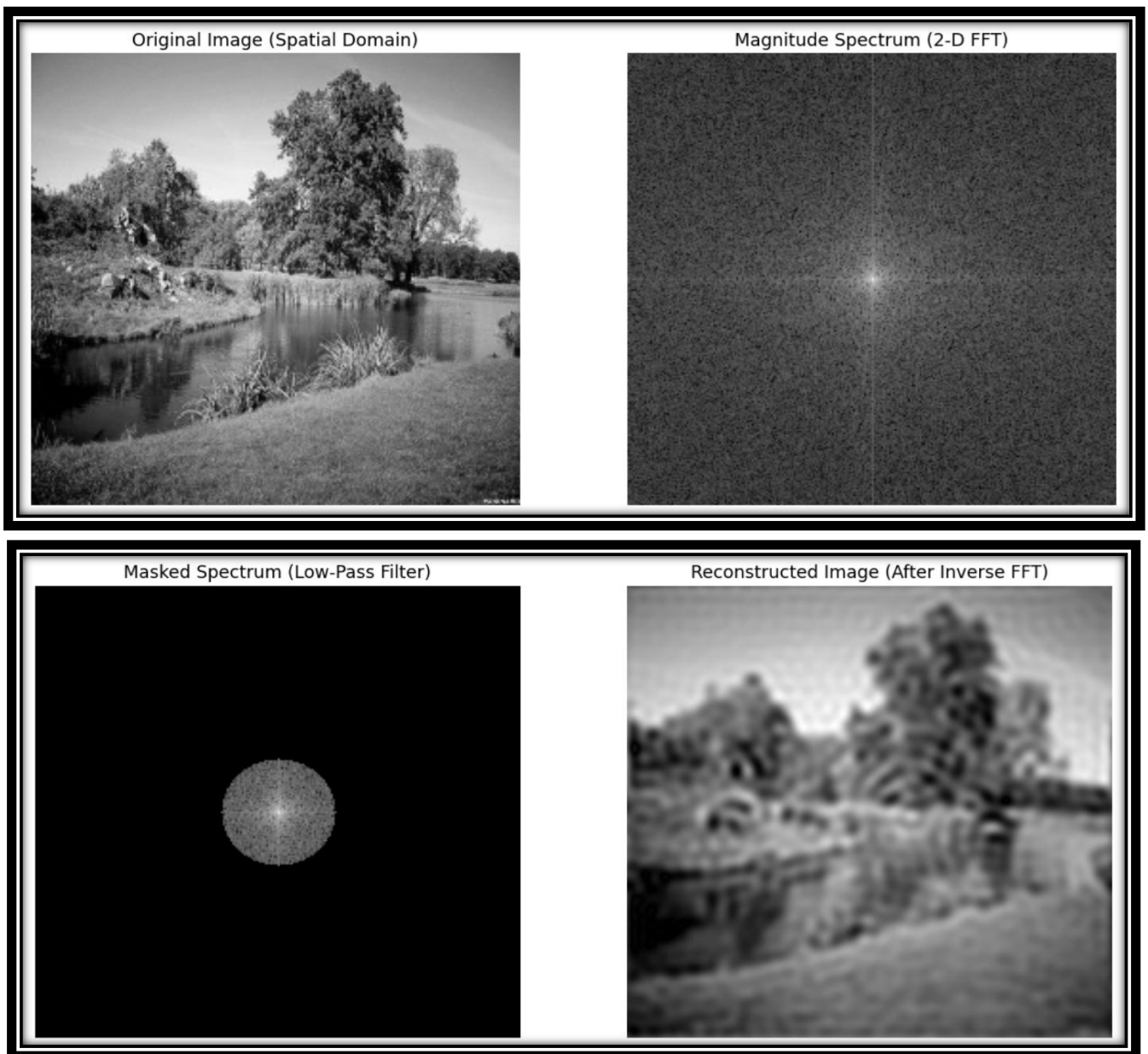
plt.subplot(2, 2, 3)
plt.imshow(masked_spectrum, cmap='gray')
plt.title('Masked Spectrum (Low-Pass Filter)')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(reconstructed, cmap='gray')
plt.title('Reconstructed Image (After Inverse FFT)')
plt.axis('off')

plt.tight_layout()
plt.show()

```

OUTPUT:



LEARNING OUTCOME:

EXPERIMENT 3

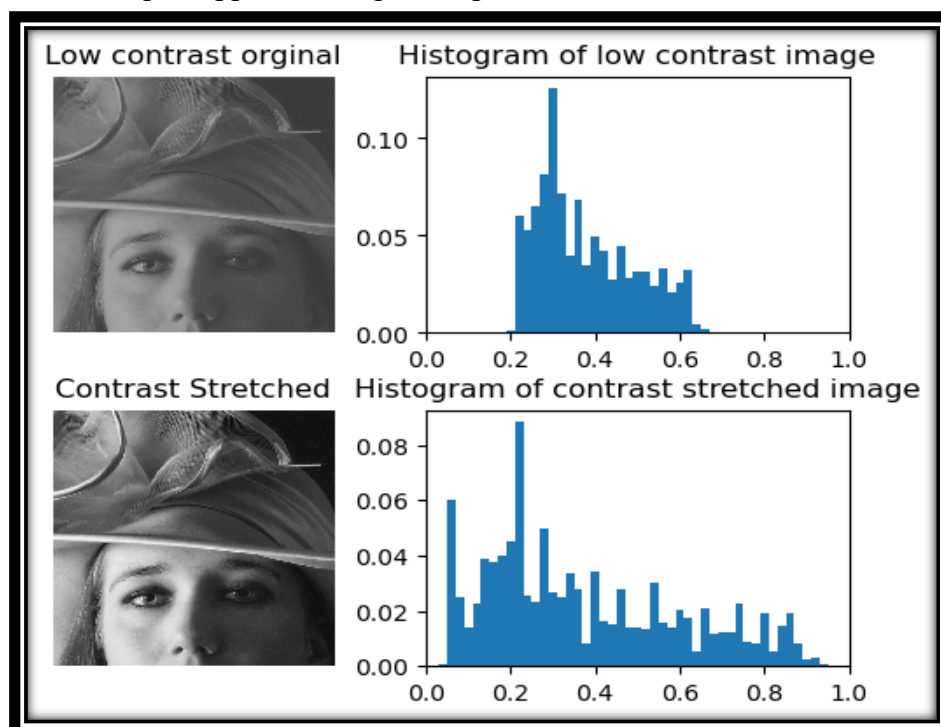
PROBLEM STATEMENT: Write a program for Contrast stretching of a low contrast image, Histogram, Histogram Equalization, and display of bit planes of an image.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Contrast Stretching: - Contrast stretching is a fundamental image enhancement technique used to improve the visibility of features in a low-contrast image by expanding the range of intensity values. In many real-world scenarios, images suffer from poor contrast due to inadequate lighting or sensor limitations, resulting in pixel values being clustered within a narrow range. This technique linearly maps the original intensity values to a wider range, typically from 0 to 255 in grayscale images. In this experiment, contrast stretching was implemented by identifying the minimum and maximum intensity values in the image and applying a normalization formula. As a result, the visual quality of the image is significantly enhanced, making details more distinguishable.

Histogram of an Image: - A histogram is a graphical representation that illustrates the distribution of pixel intensity values in an image. It provides valuable insights into the brightness, contrast, and overall tonal distribution of the image. In grayscale images, the histogram consists of 256 bins corresponding to intensity levels from 0 to 255. In this experiment, histograms were plotted for the original, contrast-stretched, and equalized images to analyse how pixel intensities are redistributed after processing. A narrow histogram indicates low contrast, whereas a widely spread histogram signifies improved contrast. Thus, histogram analysis serves as an essential tool for evaluating the effectiveness of image enhancement techniques applied during the experiment.



Histogram Equalization: - Histogram equalization is an advanced image processing technique used to enhance contrast by redistributing pixel intensity values based on the cumulative distribution function (CDF). Unlike contrast stretching, which performs linear scaling, histogram equalization is a nonlinear process that spreads frequently occurring intensity values across the entire available range. This results in a more uniform histogram and improved visibility of image details, especially in areas with poor contrast. In this experiment, histogram equalization was applied using a built-in function, and its effect was analysed by comparing the output image and its histogram with the original image. The method is particularly useful in applications such as medical imaging and satellite image analysis.

Bit Plane Slicing: - Bit plane slicing is a technique used to analyse the contribution of individual bits to the overall image representation. A grayscale image is typically composed of 8-bit pixels, where each bit represents a separate plane ranging from the least significant bit (LSB) to the most significant bit (MSB). In this experiment, the image was decomposed into its eight-bit planes using bitwise operations. Lower-order bit planes contain finer details and noise, while higher-order bit planes carry the most significant visual information. By visualizing each bit plane separately, it becomes easier to understand how image data is structured and how different levels of detail contribute to the final image.

CODE SCREENSHOTS:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg', cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image not found!")
    exit()

def contrast_stretch(img):
    min_val = np.min(img)
    max_val = np.max(img)

    stretched = ((img - min_val) / (max_val - min_val)) * 255
    return stretched.astype(np.uint8)

stretched_img = contrast_stretch(image)

def plot_histogram(img, title):
    plt.figure()
    plt.hist(img.ravel(), bins=256, range=[0,256])
    plt.title(title)
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")
    plt.show()

equalized_img = cv2.equalizeHist(image)

def bit_plane_slicing(img):
    bit_planes = []

    for i in range(8):
        bit_plane = (img >> i) & 1
        bit_planes.append(bit_plane * 255)
```

```

    return bit_planes

bit_planes = bit_plane_slicing(image)

plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(stretched_img, cmap='gray')
plt.title("Contrast Stretched")
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(equalized_img, cmap='gray')
plt.title("Histogram Equalized")
plt.axis('off')

plt.tight_layout()
plt.show()

plot_histogram(image, "Original Histogram")
plot_histogram(stretched_img, "Contrast Stretched Histogram")
plot_histogram(equalized_img, "Equalized Histogram")

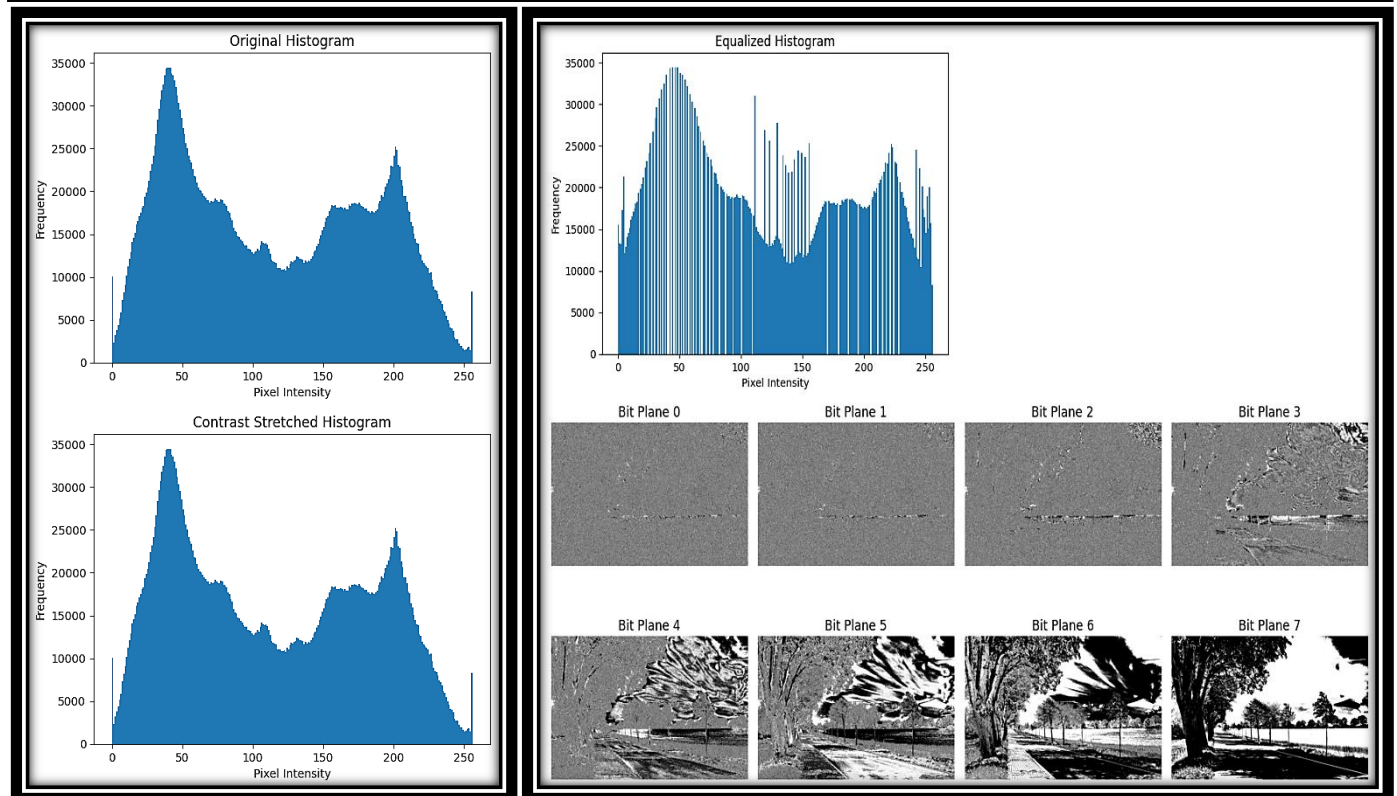
plt.figure(figsize=(12,6))

for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(bit_planes[i], cmap='gray')
    plt.title(f'Bit Plane {i}')
    plt.axis('off')

plt.tight_layout()
plt.show()

```


OUTPUT:



LEARNING OUTCOME:

EXPERIMENT 4

PROBLEM STATEMENT: Computation of Mean, Standard Deviation, Correlation coefficient of the given image.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Digital Image as a Random Variable: - A grayscale digital image can be modelled as a two-dimensional function $f(x, y)$, where x and y represent the spatial coordinates of the image and $f(x, y)$ denotes the grey level intensity of the pixel at that location.

For an 8-bit grayscale image, the intensity values range from 0 to 255.

In digital image processing, pixel intensities are often treated as random variables. This statistical interpretation allows the use of parameters such as mean, standard deviation, and correlation coefficient to analyze image characteristics.

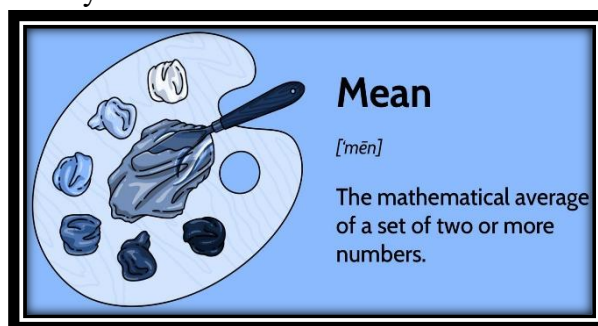
Mean of an Image: -The mean of an image represents the average grey level of all the pixels and provides an estimate of the overall brightness of the image. It is computed by summing all pixel intensities and dividing by the total number of pixels in the image.

Interpretation –

- A higher mean value indicates a brighter image.
- A lower mean value indicates a darker image.

Applications in Digital Image Processing –

- Image brightness adjustment
- Image normalization
- Histogram-based analysis



Standard Deviation of an Image: - Standard deviation measures the dispersion of pixel intensity values around the mean. It indicates how much variation exists in the image's grey levels.

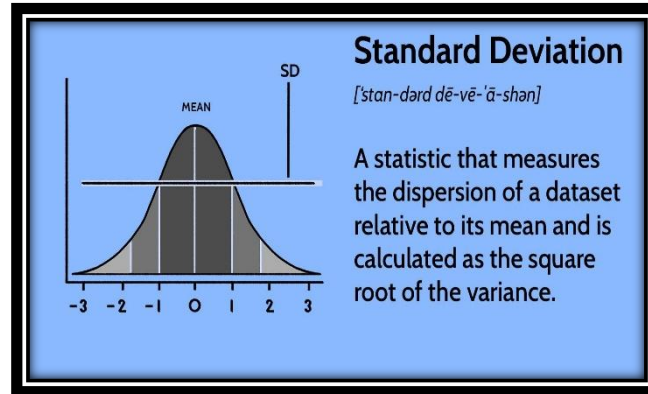
Interpretation –

- Low standard deviation corresponds to low contrast images with nearly uniform intensity.

- High standard deviation corresponds to high contrast images with widely varying intensities.

Applications in Digital Image Processing –

- Contrast evaluation
- Texture analysis
- Noise estimation



Correlation Coefficient of Images: -

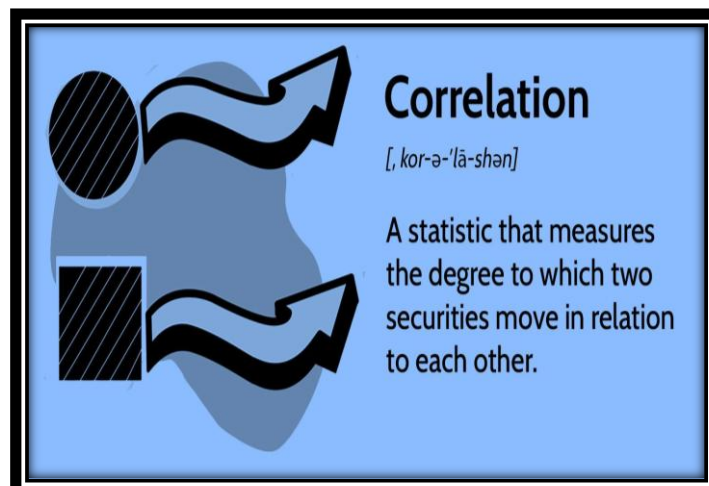
The correlation coefficient is a statistical measure used to determine the similarity between two images. It compares the relationship between corresponding pixel values of two images and produces a value between -1 and $+1$.

Interpretation of Correlation Coefficient

- $r = +1$: Perfect positive correlation
- $r = 0$: No correlation
- $r = -1$: Perfect negative correlation

Applications in Digital Image Processing

- Image similarity measurement
- Template matching
- Change detection
- Image registration



CODE SCREENSHOTS:

```
[1]
✓ 0s  import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read two grayscale images
img1 = cv2.imread('test_image.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('test_image1.jpg', cv2.IMREAD_GRAYSCALE)

# Check if images are loaded
if img1 is None or img2 is None:
    print("Error: One or both images not found")
    exit()

# Ensure both images are of same size
if img1.shape != img2.shape:
    print("Error: Images must be of the same size")
    exit()

# Display both images
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(img1, cmap='gray')
plt.title("Image 1")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(img2, cmap='gray')
plt.title("Image 2")
plt.axis('off')

plt.show()
```

```
# Mean and Standard Deviation of Image 1
mean1 = np.mean(img1)
std1 = np.std(img1)

# Mean and Standard Deviation of Image 2
mean2 = np.mean(img2)
std2 = np.std(img2)

# Flatten images for correlation
img1_flat = img1.flatten()
img2_flat = img2.flatten()

# Correlation Coefficient
correlation = np.corrcoef(img1_flat, img2_flat)[0, 1]

# Display results
print("Image 1 Mean:", mean1)
print("Image 1 Standard Deviation:", std1)
print("Image 2 Mean:", mean2)
print("Image 2 Standard Deviation:", std2)
print("Correlation Coefficient:", correlation)
```

OUTPUT:

Image 1



Image 2



Image 1 Mean: 100.24799329572099
Image 1 Standard Deviation: 60.832513938460025
Image 2 Mean: 164.30264416031653
Image 2 Standard Deviation: 39.426005265078416
Correlation Coefficient: 0.12272399887070523

LEARNING OUTCOME:

EXPERIMENT 5

PROBLEM STATEMENT: Implementation of Image Smoothing Filters (Mean and Median filtering of an image).

APPARATUS: Colab Notebook (Python), .jpg image

Image smoothing, commonly referred to as image blurring, is an essential preprocessing operation in digital image processing. The main purpose of smoothing is to reduce noise and minimize high-frequency variations such as sudden intensity changes, sharp edges, and fine textures present in an image. During image acquisition, unwanted disturbances like Gaussian noise or impulse noise may degrade image quality. Smoothing filters help in reducing these irregularities and producing a visually consistent image. In Artificial Intelligence and Machine Learning applications, image smoothing plays an important role in simplifying visual data before performing tasks such as segmentation, edge detection, and object recognition. By suppressing small fluctuations, the algorithm can focus more effectively on meaningful structures rather than pixel-level noise.

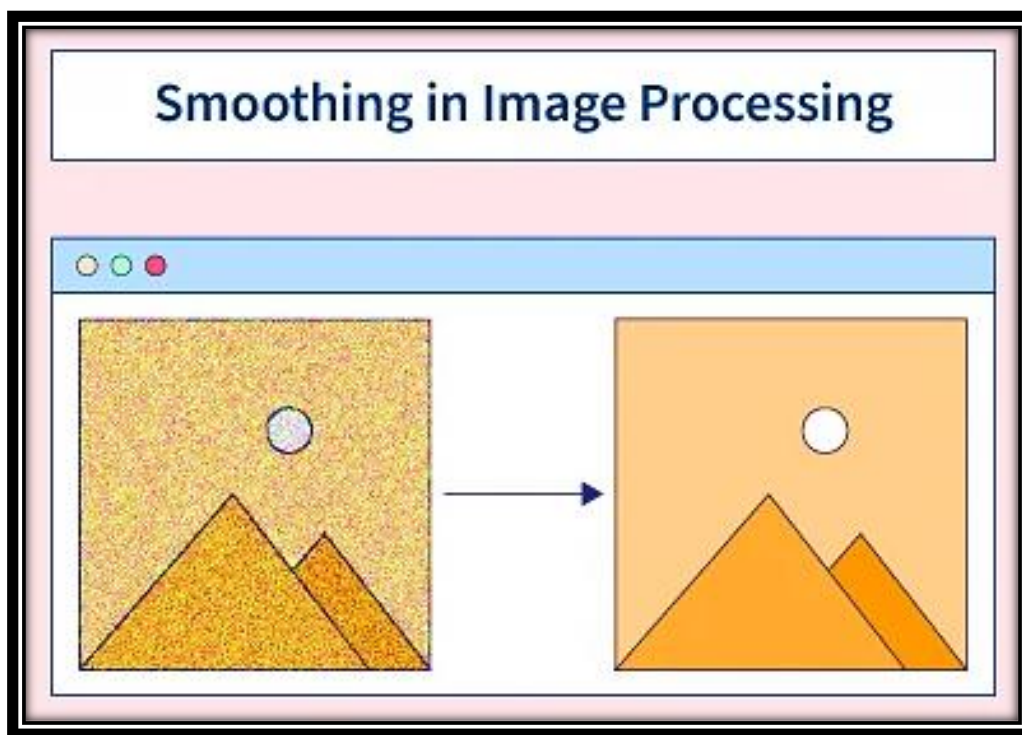


Image smoothing is typically implemented using convolution, a process in which a small matrix called a kernel or filter moves across the image. At each position, the kernel interacts with neighboring pixel values to compute a new intensity value for the central pixel. This neighborhood-based computation modifies the image gradually and results in noise reduction. Among various smoothing approaches, Mean Filtering and Median Filtering are two widely used techniques due to their simplicity and effectiveness.

Mean Filtering (Linear Filter) - Mean filtering is one of the simplest image smoothing techniques and belongs to the category of linear filters because the output pixel value is calculated as a linear combination of neighboring pixels. In this method, a small window (for

example, 3×3 or 5×5) slides across the image, and the center pixel is replaced with the average intensity value of all pixels inside the window. For a 3×3 kernel, the sum of nine-pixel values is computed and divided by nine to obtain the new pixel value.

This averaging process reduces intensity variations and smooths minor noise present in the image. However, since all pixels contribute equally, important details such as edges and fine textures also get blurred. As a result, the image may appear softer or slightly out of focus. Another limitation is its poor performance against impulse noise, where extreme pixel values are averaged rather than eliminated, causing noise to spread instead of being removed.

Median Filtering (Non-Linear Filter) - Median filtering is a non-linear image smoothing technique that is particularly effective for removing salt-and-pepper noise, which appears as random black and white pixels in an image. Unlike mean filtering, this method does not compute an average. Instead, all pixel values within the selected neighborhood are sorted in ascending order, and the median (middle value) replaces the center pixel.

Because the median operation is resistant to extreme values, noisy pixels that significantly differ from their neighbors are automatically discarded during sorting. This makes the median filter highly effective for impulse noise removal while preserving important structural features. One of its major advantages is edge preservation; transitions between bright and dark regions remain sharp compared to the blurred boundaries produced by mean filtering. Although computationally more intensive, median filtering provides superior visual quality in many practical image restoration tasks.

Effect of Kernel Size on Smoothing - The degree of image smoothing depends largely on the size of the kernel used during filtering. Smaller kernels, such as 3×3 , perform mild smoothing and remove only minor noise while maintaining most image details. They are suitable when detail preservation is important. As the kernel size increases to 5×5 or 9×9 , the filter considers a larger neighborhood, resulting in stronger blurring effects.

Larger kernels significantly reduce noise but may also remove fine features and distort small objects within the image. Excessive smoothing can lead to loss of important information, making objects appear unclear or merged with the background. Therefore, selecting an appropriate kernel size requires balancing noise reduction with detail preservation depending on the application requirements.

Practical Applications in Artificial Intelligence - In modern AI and computer vision systems, image smoothing is primarily used as a preprocessing step rather than a final operation. Many edge detection algorithms, such as the Canny Edge Detector, apply smoothing beforehand to prevent noise from being incorrectly detected as edges. Smoothing also helps simplify image features by removing textures and small variations, allowing machine learning models to focus on dominant shapes and object boundaries. This improves feature extraction and enhances model robustness. Consequently, smoothing techniques play a crucial role in improving the accuracy and stability of image analysis pipelines used in applications like medical imaging, autonomous systems, and object recognition.

CODE SCREENSHOTS:

[1]
✓ 12s



```
!pip install opencv-python
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image_path = "/content/test_image.jpg"

image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    raise Exception("Image not found. Upload image first.")

mean_filtered = cv2.blur(image, (3,3))

gaussian_filtered = cv2.GaussianBlur(image, (5,5), 1.5)

median_filtered = cv2.medianBlur(image, 3)

titles = ['Original Image', 'Mean Filter', 'Gaussian Filter', 'Median Filter']
images = [image, mean_filtered, gaussian_filtered, median_filtered]

plt.figure(figsize=(12,8))

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')

plt.show()
```

OUTPUT:

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)

Original Image



Mean Filter



Gaussian Filter



Median Filter



LEARNING OUTCOME:

EXPERIMENT 6

PROBLEM STATEMENT: Implementation of Image sharpening filters and Edge Detection using Gradient Filters.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Image sharpening and edge detection are fundamental operations in digital image processing that help improve visual clarity by emphasizing important structural information in an image. Unlike smoothing techniques, which reduce noise by softening intensity variations, sharpening enhances sudden intensity changes, thereby making object boundaries and fine details more prominent. Edge detection methods based on gradients are particularly useful for identifying regions where pixel intensity varies significantly.

Image sharpening techniques operate by increasing the contrast between neighbouring pixels. Areas with abrupt intensity variation usually represent edges or detailed features. By amplifying these variations, sharpening filters make image features clearer and more distinguishable. Most sharpening operations are performed in the spatial domain, where a small matrix called a kernel or mask is systematically applied across the image to modify pixel values based on their surrounding neighbourhood.

Many sharpening methods rely on derivative concepts from calculus. A derivative represents the rate at which a quantity changes. In digital images, pixel intensity can be considered a function of spatial position. The first-order derivative is primarily used to detect edges, while the second-order derivative enhances finer details and sharp transitions even more strongly.

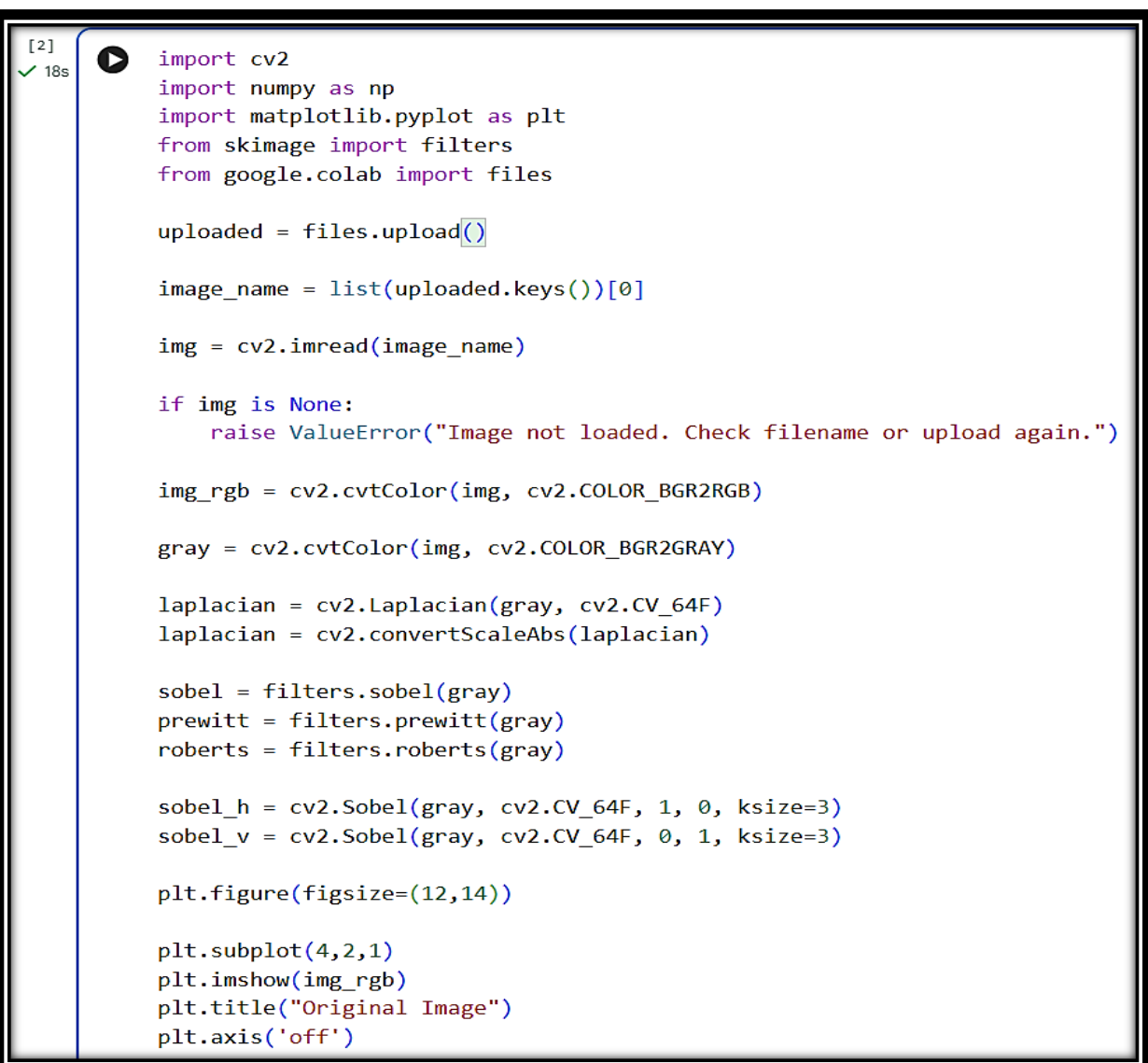
One commonly used sharpening technique is the Laplacian filter, which is derived from the second-order derivative. The Laplacian operator evaluates how intensity changes with respect to neighbouring pixels in all directions simultaneously. Pixels whose intensities differ greatly from their surroundings generate strong responses. Typically, the Laplacian output is combined with the original image to enhance edge contrast and improve detail visibility. However, since this process can also magnify noise, smoothing operations are sometimes performed before applying sharpening.



Gradient-based edge detection methods are based on first-order derivatives. The gradient measures the rate of intensity change along horizontal and vertical directions. A large intensity variation in any direction indicates the possible presence of an edge. Mathematically, the image gradient consists of two components: the horizontal component (x-direction) and the vertical component (y-direction). These components are calculated using convolution masks applied to the image. The overall edge strength is determined by computing the gradient magnitude, which is obtained from the square root of the sum of squared horizontal and vertical components. Higher gradient values correspond to stronger edges.

Several gradient operators are widely used for edge detection, including Sobel, Prewitt, and Roberts filters. The Sobel operator employs two 3×3 kernels to detect edges in horizontal and vertical orientations and assigns greater importance to central pixels, improving noise resistance. The Prewitt operator functions similarly but uses equal weighting across the kernel. The Roberts operator uses a smaller 2×2 mask, making it computationally simple but more sensitive to noise. All these operators provide approximations of the first derivative of image intensity to identify edges effectively.

CODE SCREENSHOTS:



```
[2]
✓ 18s
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import filters
from google.colab import files

uploaded = files.upload()

image_name = list(uploaded.keys())[0]

img = cv2.imread(image_name)

if img is None:
    raise ValueError("Image not loaded. Check filename or upload again.")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

sobel = filters.sobel(gray)
prewitt = filters.prewitt(gray)
roberts = filters.roberts(gray)

sobel_h = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_v = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

plt.figure(figsize=(12,14))

plt.subplot(4,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
```

[2]
✓ 18s



```
plt.subplot(4,2,2)
plt.imshow(gray, cmap='gray')
plt.title("Gray Image")
plt.axis('off')

plt.subplot(4,2,3)
plt.imshow(laplacian, cmap='gray')
plt.title("Laplacian")
plt.axis('off')

plt.subplot(4,2,4)
plt.imshow(sobel, cmap='gray')
plt.title("Sobel")
plt.axis('off')

plt.subplot(4,2,5)
plt.imshow(rewitt, cmap='gray')
plt.title("Prewitt")
plt.axis('off')

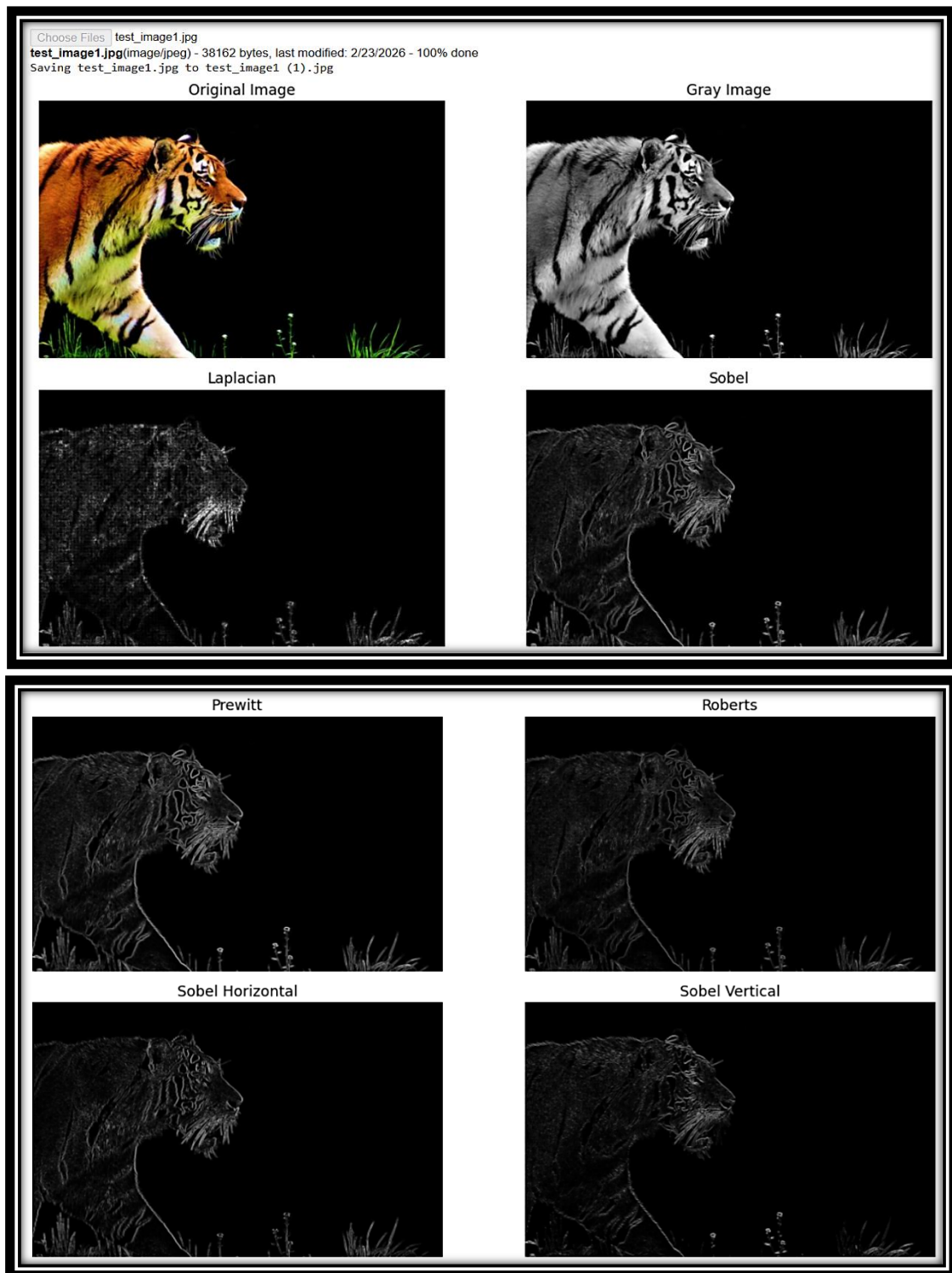
plt.subplot(4,2,6)
plt.imshow(roberts, cmap='gray')
plt.title("Roberts")
plt.axis('off')

plt.subplot(4,2,7)
plt.imshow(np.abs(sobel_h), cmap='gray')
plt.title("Sobel Horizontal")
plt.axis('off')

plt.subplot(4,2,8)
plt.imshow(np.abs(sobel_v), cmap='gray')
plt.title("Sobel Vertical")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



LEARNING OUTCOME:

EXPERIMENT 7

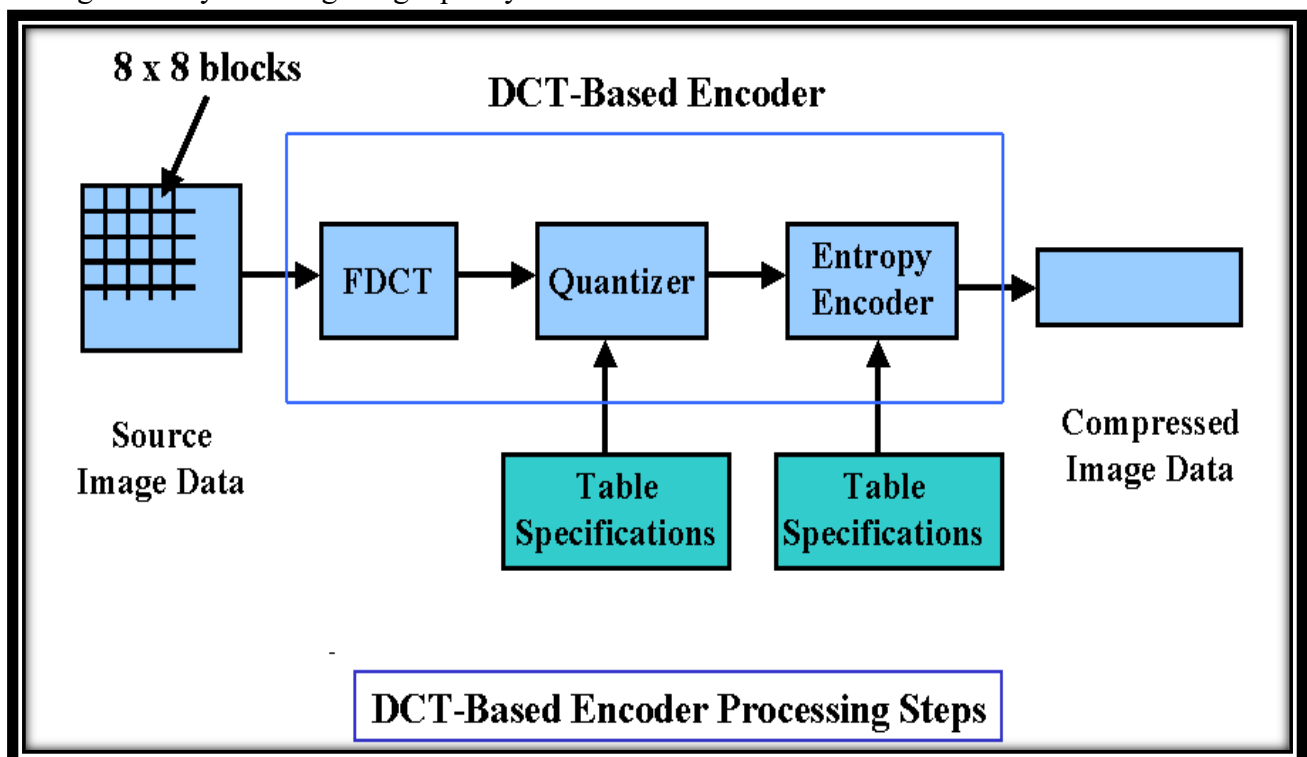
PROBLEM STATEMENT: Implementation of Image Compression by DCT, DPCM, HUFFMAN coding.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Discrete Cosine Transform (DCT) - The Discrete Cosine Transform (DCT) is a frequency-domain transformation technique widely used in image compression. It converts spatial pixel intensity values into frequency components, separating low-frequency information from high-frequency details. Most visual information in an image is concentrated in low-frequency components, allowing high-frequency coefficients to be discarded with minimal perceptual loss. DCT operates typically on small blocks such as 8×8 pixels, reducing redundancy and enabling efficient compression. This principle forms the foundation of standard compression techniques like JPEG, where compression is achieved by retaining significant coefficients while suppressing less important ones.

Differential Pulse Code Modulation (DPCM) - Differential Pulse Code Modulation (DPCM) is a predictive coding technique that compresses data by encoding the difference between consecutive pixel values instead of absolute intensities. Since neighboring pixels in images are highly correlated, the difference signal usually contains smaller values requiring fewer bits for representation. A predictor estimates the current pixel using previously reconstructed pixels, and only the prediction error is transmitted or stored. During decoding, the original signal is reconstructed by adding the prediction error to the predicted value. DPCM effectively reduces spatial redundancy and improves compression efficiency without significantly affecting image quality.





Huffman Coding - Huffman coding is a lossless entropy encoding technique used to reduce data size based on symbol probability. It assigns shorter binary codes to frequently occurring symbols and longer codes to less frequent ones, minimizing the average code length. A binary tree called the Huffman tree is constructed using symbol frequencies, ensuring optimal prefix-free encoding. Since no information is lost during encoding or decoding, Huffman coding preserves data integrity while improving storage efficiency. It is commonly used as the final stage in many compression systems, including JPEG, to further compress already processed image data.

Image Compression Concept - Image compression is the process of reducing the amount of data required to represent an image while maintaining acceptable visual quality. It removes redundancy and irrelevant information using transformation, prediction, and encoding techniques. Compression methods are broadly classified into lossless and lossy techniques depending on whether information is discarded. In this experiment, DCT reduces spatial redundancy, DPCM minimizes inter-pixel correlation, and Huffman coding optimizes bit representation. The combined approach achieves efficient compression by reducing storage requirements and transmission bandwidth while preserving essential image characteristics.

CODE SCREENSHOTS:

```
[1] !pip -q install opencv-python

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from collections import Counter
import heapq

print("Upload a grayscale or color image")
uploaded = files.upload()

img_name = list(uploaded.keys())[0]
img = cv2.imread(img_name, cv2.IMREAD_GRAYSCALE)

img = cv2.resize(img, (256,256))
img = img.astype(np.float32)

plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.show()
```

[1]



```
block_size = 8
h, w = img.shape
dct_image = np.zeros_like(img)

for i in range(0, h, block_size):
    for j in range(0, w, block_size):
        block = img[i:i+block_size, j:j+block_size]
        dct_block = cv2.dct(block)

        mask = np.zeros_like(dct_block)
        mask[:4,:4] = 1
        dct_block = dct_block * mask

        dct_image[i:i+block_size, j:j+block_size] = dct_block

reconstructed_dct = np.zeros_like(img)

for i in range(0, h, block_size):
    for j in range(0, w, block_size):
        block = dct_image[i:i+block_size, j:j+block_size]
        reconstructed_dct[i:i+block_size, j:j+block_size] = cv2.idct(block)

dpcm_encoded = np.zeros_like(reconstructed_dct)
prediction = np.zeros_like(reconstructed_dct)

for i in range(h):
    for j in range(w):
        if j == 0:
            pred = 0
        else:
            pred = reconstructed_dct[i, j-1]

        prediction[i,j] = pred
        dpcm_encoded[i,j] = reconstructed_dct[i,j] - pred

dpcm_decoded = np.zeros_like(dpcm_encoded)
```

[1]



```
for i in range(h):
    for j in range(w):
        if j == 0:
            pred = 0
        else:
            pred = dpcm_decoded[i,j-1]

        dpcm_decoded[i,j] = dpcm_encoded[i,j] + pred

data = dpcm_encoded.flatten().astype(int)

class Node:
    def __init__(self, freq, symbol=None, left=None, right=None):
        self.freq = freq
        self.symbol = symbol
        self.left = left
        self.right = right
    def __lt__(self, other):
        return self.freq < other.freq

freq = Counter(data)
heap = [Node(f, s) for s, f in freq.items()]
heapq.heapify(heap)

while len(heap) > 1:
    n1 = heapq.heappop(heap)
    n2 = heapq.heappop(heap)
    merged = Node(n1.freq+n2.freq, left=n1, right=n2)
    heapq.heappush(heap, merged)

root = heap[0]
```

[1]



```
huffman_codes = {}
def generate_codes(node, code=""):
    if node is None:
        return
    if node.symbol is not None:
        huffman_codes[node.symbol] = code
    generate_codes(node.left, code+"0")
    generate_codes(node.right, code+"1")

generate_codes(root)
encoded_bits = "".join(huffman_codes[val] for val in data)
original_bits = img.size * 8
compressed_bits = len(encoded_bits)

compression_ratio = original_bits / compressed_bits

print("\n==== Compression Results =====")
print("Original Size (bits):", original_bits)
print("Compressed Size (bits):", compressed_bits)
print("Compression Ratio:", round(compression_ratio,2))

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(reconstructed_dct, cmap='gray')
plt.title("After DCT Compression")

plt.subplot(1,3,3)
plt.imshow(dpcm_decoded, cmap='gray')
plt.title("After DPCM + Huffman")

plt.show()
```

OUTPUT:

...

Upload a grayscale or color image

Choose Files

test.jpg

test.jpg(image/jpeg) - 856317 bytes, last modified: 3/2/2026 - 100% done

Saving test.jpg to test.jpg

Original Image

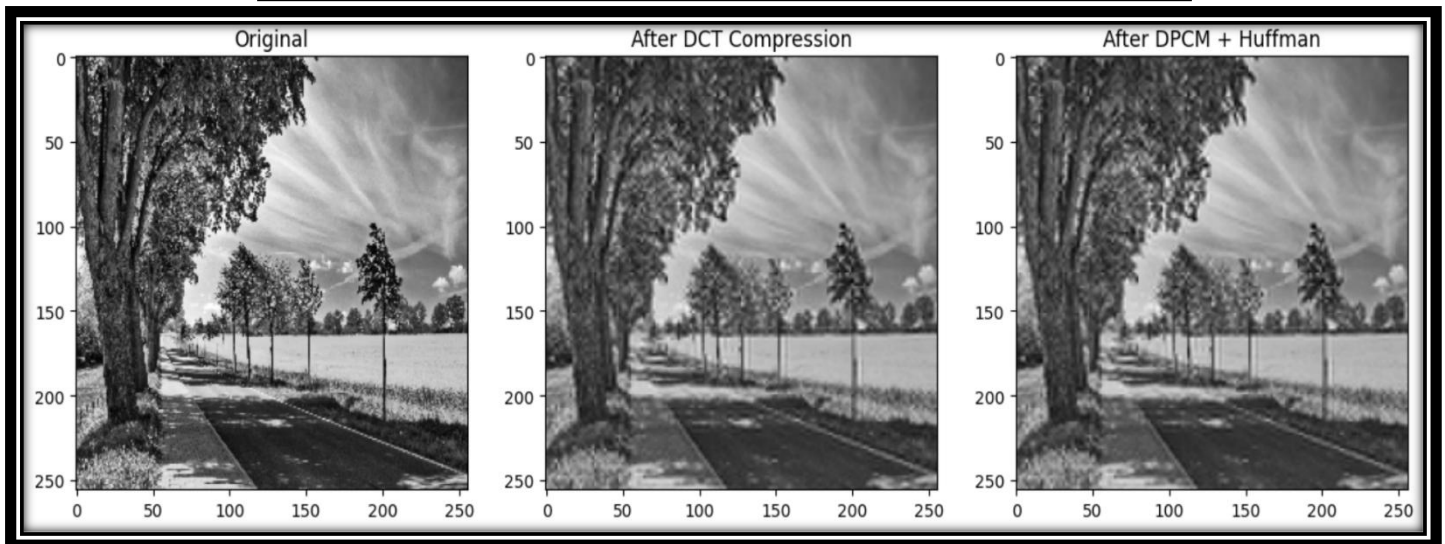


==== Compression Results ====

Original Size (bits): 524288

Compressed Size (bits): 387778

Compression Ratio: 1.35



LEARNING OUTCOME:

EXPERIMENT 8

PROBLEM STATEMENT: Implementation of image restoring techniques.

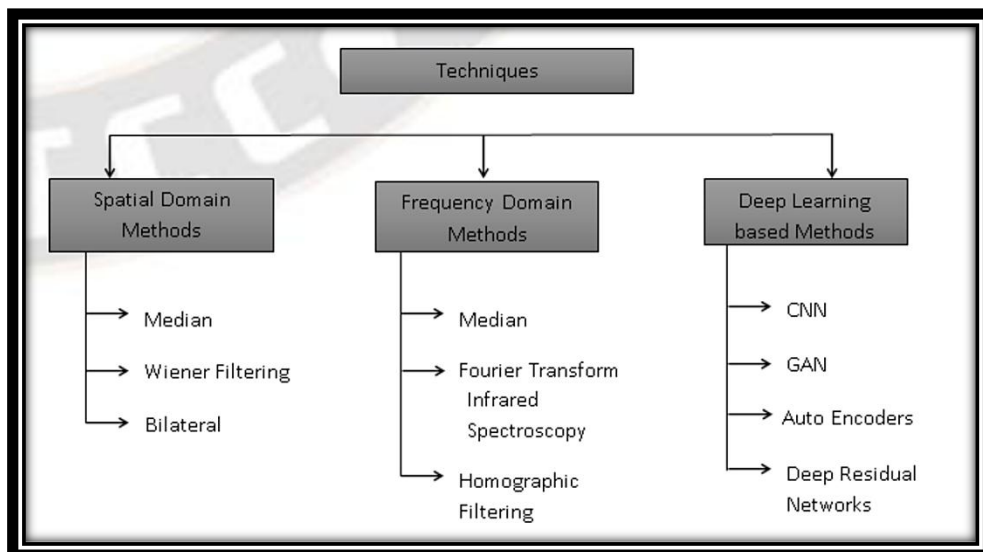
APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Introduction to Image Restoration: - Image restoration is a fundamental concept in digital image processing that focuses on recovering an original, high-quality image from a degraded version. Degradation may occur due to various factors such as sensor noise, transmission errors, motion blur, or environmental conditions. Unlike image enhancement, which is subjective, restoration is based on mathematical and statistical models of degradation. The primary objective is to reconstruct the original image as accurately as possible by reducing noise and distortion. Restoration techniques are widely used in fields such as medical imaging, satellite imaging, surveillance systems, and computer vision applications.

Noise Models in Images: - Noise is an unwanted disturbance that affects image quality and obscures important details. In this experiment, two common noise models are considered: Gaussian noise and Salt-and-Pepper noise. Gaussian noise is statistical in nature and follows a normal distribution, often introduced during image acquisition due to electronic circuit fluctuations. Salt-and-Pepper noise, also known as impulse noise, appears as random black and white pixels caused by transmission errors or faulty sensors. Understanding the type of noise present in an image is crucial, as different restoration techniques are designed to handle specific noise characteristics effectively.

Linear Filtering Techniques: - Linear filtering techniques, such as the Mean filter and Gaussian filter, are commonly used for smoothing and reducing noise in images. The Mean filter replaces each pixel value with the average of its neighboring pixels, effectively reducing random variations but often causing blurring of edges. The Gaussian filter improves upon this by applying a weighted average, where pixels closer to the center have higher importance. This results in more natural smoothing while preserving some structural details. However, both filters are less effective in handling impulse noise and may lead to loss of fine image features.



Non-Linear Filtering Techniques: - Non-linear filters, particularly the Median filter, are highly effective in removing impulse noise such as Salt-and-Pepper noise. Unlike linear filters, the Median filter replaces each pixel with the median value of its neighborhood rather than the average. This approach preserves edges and fine details while eliminating extreme noise values. Because it does not rely on averaging, it avoids the blurring effect commonly observed in linear filters. The Median filter is widely used in practical applications where maintaining edge sharpness is critical, making it a preferred choice for impulse noise removal.

Wiener Filter and Advanced Restoration: - The Wiener filter is an advanced image restoration technique based on statistical estimation theory. It aims to minimize the mean square error between the restored image and the original image by considering both the degradation function and noise characteristics. Unlike simple filters, the Wiener filter adapts to local image variance, providing a balance between noise reduction and detail preservation. It is particularly effective for images corrupted by Gaussian noise and blur. Due to its optimal restoration capability, the Wiener filter is widely used in scientific imaging, medical diagnostics, and signal processing applications.

CODE SCREENSHOTS:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import wiener

image = cv2.imread('/content/test_image.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

image = image / 255.0

mean = 0
sigma = 0.05
gaussian_noise = np.random.normal(mean, sigma, image.shape)
noisy_gaussian = image + gaussian_noise
noisy_gaussian = np.clip(noisy_gaussian, 0, 1)

salt_prob = 0.02
pepper_prob = 0.02

noisy_sp = np.copy(image)
num_salt = np.ceil(salt_prob * image.size)
coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
noisy_sp[coords[0], coords[1]] = 1

num_pepper = np.ceil(pepper_prob * image.size)
coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]
noisy_sp[coords[0], coords[1]] = 0
```



```
mean_filtered = cv2.blur(noisy_gaussian, (5, 5))

median_filtered = cv2.medianBlur((noisy_sp * 255).astype(np.uint8), 5)

gaussian_filtered = cv2.GaussianBlur(noisy_gaussian, (5, 5), 0)

wiener_filtered = wiener(noisy_gaussian, (5, 5))

plt.figure(figsize=(12, 10))

plt.subplot(3, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(3, 3, 2)
plt.imshow(noisy_gaussian, cmap='gray')
plt.title("Gaussian Noise")
plt.axis('off')

plt.subplot(3, 3, 3)
plt.imshow(noisy_sp, cmap='gray')
plt.title("Salt & Pepper Noise")
plt.axis('off')

plt.subplot(3, 3, 4)
plt.imshow(mean_filtered, cmap='gray')
plt.title("Mean Filter")
plt.axis('off')

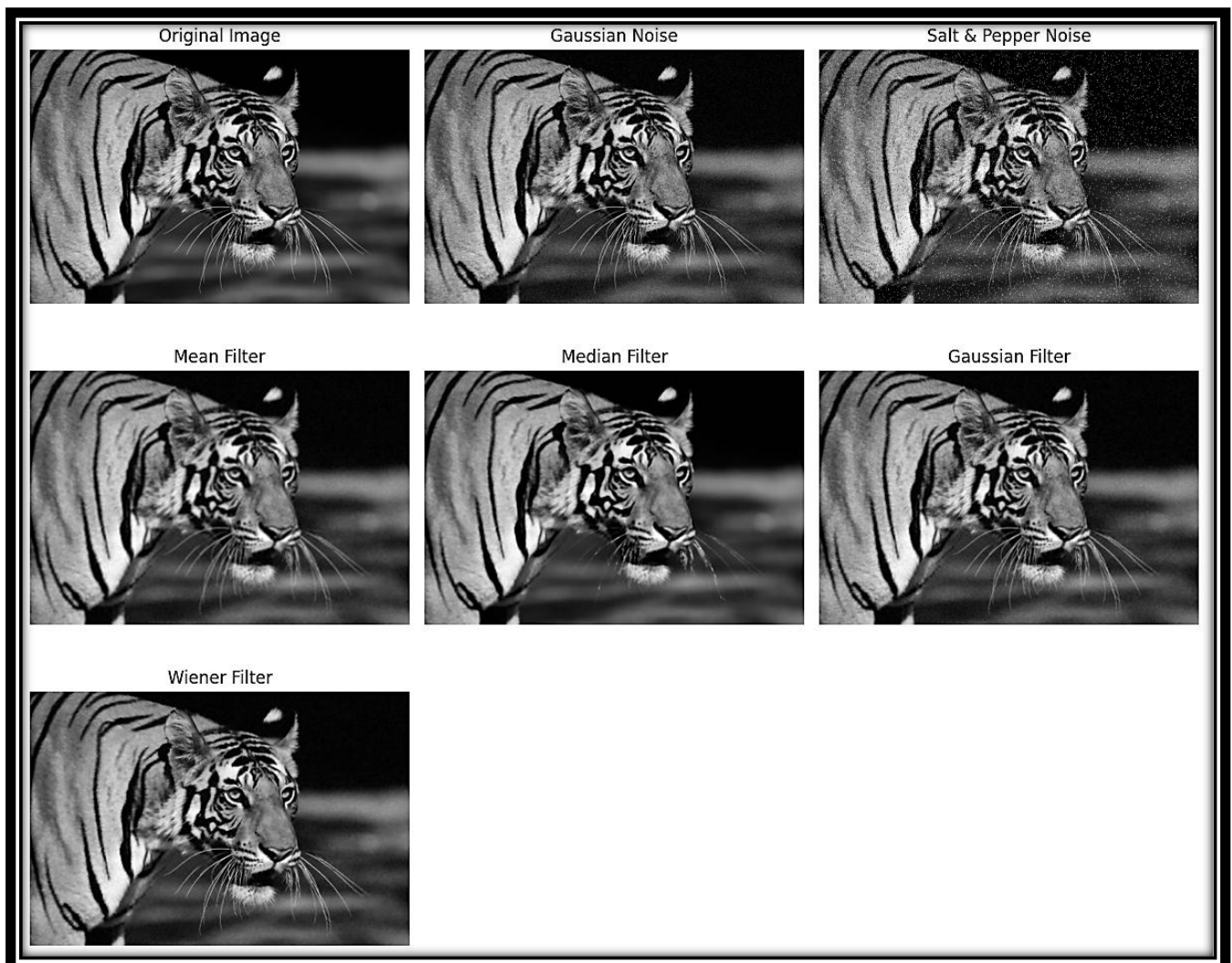
plt.subplot(3, 3, 5)
plt.imshow(median_filtered, cmap='gray')
plt.title("Median Filter")
```

```
plt.subplot(3, 3, 6)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title("Gaussian Filter")
plt.axis('off')

plt.subplot(3, 3, 7)
plt.imshow(wiener_filtered, cmap='gray')
plt.title("Wiener Filter")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



LEARNING OUTCOME:

EXPERIMENT 9

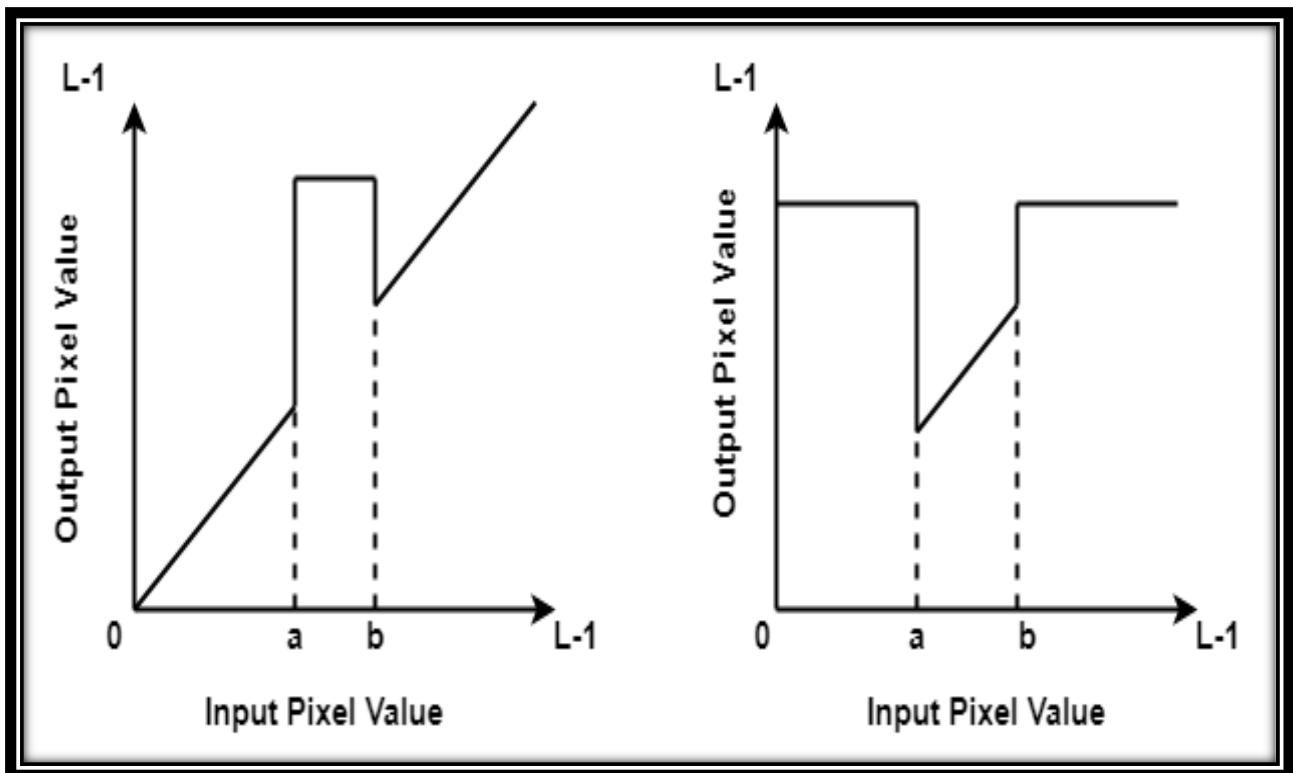
PROBLEM STATEMENT: Implementation of Image Intensity slicing technique for image enhancement.

APPARATUS: Colab Notebook (Python), .jpg image

THEORY:

Image Intensity Slicing: - Image intensity slicing is an image enhancement technique used to highlight specific ranges of intensity values within an image. It is particularly useful when certain features of interest lie within a known intensity range and need to be emphasized for better visualization. This technique involves selecting a range of gray-level values and assigning them a higher intensity, while suppressing or modifying other intensity levels. In this experiment, intensity slicing was implemented by defining lower and upper threshold values, and pixels within this range were highlighted. This method is widely used in applications such as medical imaging, remote sensing, and industrial inspection.

Intensity Slicing Without Background: - In intensity slicing without background, only the selected intensity range is preserved and highlighted, while all other pixel values are suppressed, usually set to zero or a constant value. This results in a binary-like image where only the regions of interest are visible, making it easier to isolate specific features. In this experiment, pixels falling within the chosen intensity range were assigned a high value (e.g., 255), and all other pixels were set to zero. This approach is particularly useful for feature extraction and segmentation tasks, as it removes unnecessary background information and focuses solely on the desired intensity levels.



Intensity Slicing with Background: - In intensity slicing with background preservation, the selected intensity range is enhanced while the remaining pixel values are retained in their original form. This allows the image to maintain its overall structure while still emphasizing the region of interest. In this experiment, pixels within the specified intensity range were assigned a higher value, while all other pixels remained unchanged. This technique is useful when contextual information is important for interpretation, as it provides a balance between enhancement and preservation. It is commonly applied in applications such as medical diagnosis, where highlighting specific tissues while retaining surrounding structures is essential.

CODE SCREENSHOTS:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg', cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image not found!")
    exit()

def intensity_slicing(img, lower, upper, high_value=255, background=0):
    sliced = np.copy(img)

    mask = (img >= lower) & (img <= upper)
    sliced[mask] = high_value
    sliced[~mask] = background

    return sliced

def intensity_slicing_with_bg(img, lower, upper, high_value=255):
    sliced = np.copy(img)
    mask = (img >= lower) & (img <= upper)
    sliced[mask] = high_value
    return sliced

lower_limit = 100
upper_limit = 180

slice_without_bg = intensity_slicing(image, lower_limit, upper_limit)
slice_with_bg = intensity_slicing_with_bg(image, lower_limit, upper_limit)
```

```
plt.figure(figsize=(10,6))

plt.subplot(1,3,1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(slice_without_bg, cmap='gray')
plt.title("Slicing Without Background")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(slice_with_bg, cmap='gray')
plt.title("Slicing With Background")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



LEARNING OUTCOME:

EXPERIMENT 10

PROBLEM STATEMENT: Study and implement Canny edge detection algorithms on images and compare it with existing edge detection algorithms.

APPARATUS: Colab Notebook (Python), .jpg image

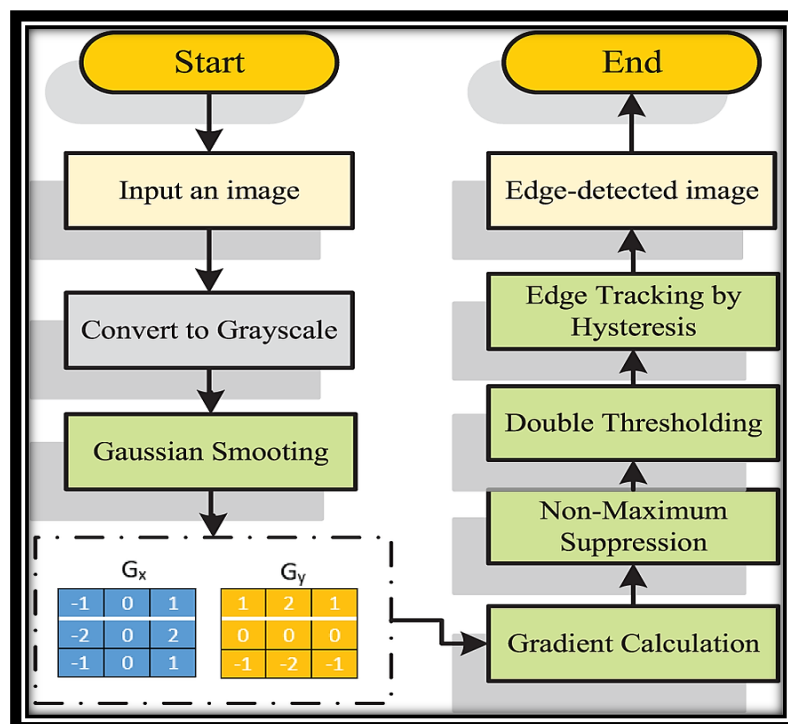
THEORY:

Edge Detection and Canny Algorithm: -

Edge detection is a fundamental operation in digital image processing used to identify sharp discontinuities in intensity values, which typically correspond to object boundaries. Among various techniques, the Canny edge detection algorithm is considered one of the most effective due to its ability to detect true edges while minimizing noise and false detections. It involves multiple stages, including Gaussian smoothing for noise reduction, gradient calculation, non-maximum suppression, and double thresholding with edge tracking. In this experiment, the Canny algorithm was implemented to extract edges from an image, demonstrating its capability to produce thin, well-defined, and continuous edges compared to other traditional methods.

Sobel Edge Detection: -

The Sobel edge detection method is a first-order derivative-based technique used to detect edges by calculating the gradient of image intensity in both horizontal and vertical directions. It uses convolution kernels to emphasize regions of high spatial frequency, which correspond to edges. In this experiment, Sobel operators were applied separately in the x and y directions, and the gradient magnitude was computed to obtain the final edge map. Although the Sobel operator is simple and computationally efficient, it is sensitive to noise and may produce thicker edges. It provides directional information but lacks the refinement and accuracy of more advanced techniques like the Canny algorithm.



Laplacian Edge Detection: -

The Laplacian edge detection technique is a second-order derivative method that detects edges by identifying regions of rapid intensity change. It calculates the Laplacian of the image, which highlights areas where the intensity changes sharply. Unlike Sobel, it does not provide directional information and is highly sensitive to noise, often resulting in noisy edge maps. In this experiment, the Laplacian operator was applied to the grayscale image to observe its edge detection performance. Although it is simple to implement, the output may contain unwanted details and requires preprocessing such as smoothing. It is generally less robust compared to the Canny edge detection algorithm.

Comparison of Edge Detection Techniques: -

In this experiment, a comparative analysis of Canny, Sobel, and Laplacian edge detection techniques was performed to evaluate their effectiveness. The Canny algorithm produced the most accurate and well-defined edges due to its multi-stage processing and noise reduction capability. Sobel provided directional edge information but resulted in thicker and less precise edges. Laplacian detected edges in all directions but was highly sensitive to noise, leading to less reliable results. Overall, Canny edge detection outperformed the other methods in terms of accuracy, noise suppression, and edge continuity, making it the preferred choice for most real-world image processing applications.

CODE SCREENSHOTS:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/image.jpg', cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image not found!")
    exit()

blurred = cv2.GaussianBlur(image, (5,5), 1.4)

canny_edges = cv2.Canny(blurred, threshold1=100, threshold2=200)

sobelx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)

sobel_edges = cv2.magnitude(sobelx, sobely)
sobel_edges = np.uint8(np.absolute(sobel_edges))

laplacian_edges = cv2.Laplacian(image, cv2.CV_64F)
laplacian_edges = np.uint8(np.absolute(laplacian_edges))

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(canny_edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
```

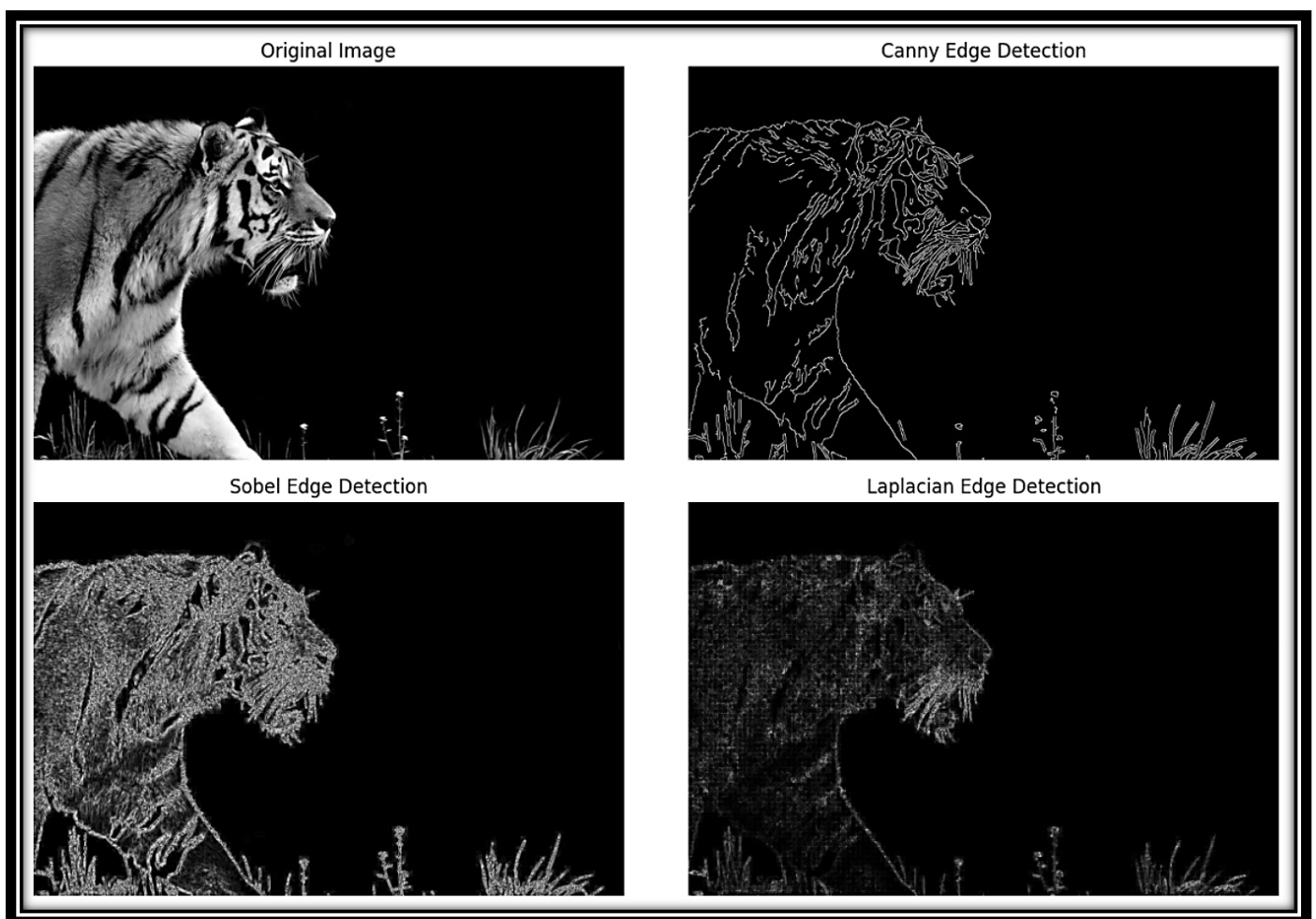


```
plt.subplot(2,2,3)
plt.imshow(sobel_edges, cmap='gray')
plt.title("Sobel Edge Detection")
plt.axis('off')

plt.subplot(2,2,4)
plt.imshow(laplacian_edges, cmap='gray')
plt.title("Laplacian Edge Detection")
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



LEARNING OUTCOME: